LAST CALL

Game Design Document — First Draft

Course: Multi-Agent AI for Game Development (ELVTR)

Assignment: #01 — First Draft of GDD

Author: Juan Diego Lugo

Date: 22 July 2026

---

1. Executive Summary

Last Call is a single-player noir detective game played entirely through interrogation. At 2:00 AM, singer Vera Lane is found dead in her dressing room at the Gilded Lily jazz club. The doors were locked at midnight; five people were inside. The player is the detective, and the five suspects are AI agents — each with its own personality, alibi, secrets, and reasons to lie.

The player has until dawn (30 interrogation actions) to question suspects, catch contradictions between their testimonies, confront them with evidence, and finally accuse the killer.

- **Win condition:** Accuse the correct suspect while presenting at least 2 pieces of supporting evidence — *"Case Closed."*
- **Loss conditions:** Accuse the wrong suspect (*"Wrongful Arrest"*), or run out of actions before accusing anyone (*"Cold Case"*).

Every playthrough uses a generated case file, so the killer, motive, and contradictions differ between runs. The game is text-based with static character portraits, built by one developer over the remaining weeks of the course: a Python backend calling the Claude API, with a minimal browser chat interface.

Elevator pitch: *Her Story* meets *Ace Attorney*, except the suspects are live AI agents who improvise under pressure — and sometimes lie badly.

---

2. Game Mechanics (player-facing actions and loop)

2.1 Core Loop

1. **Choose a suspect** from the club's back room (5 portraits, each showing a visible stress level).
2. **Interrogate** — type a free-form question. The suspect answers in character, in 1–3 sentences. Each question costs 1 action.
3. **Read the notebook** — the detective's notebook automatically records facts, highlights when two statements clash, and turns confirmed clashes into **Leads**.
4. **Press** — spend 1 action to confront a suspect with a Lead or evidence

item. A pressed suspect gets visibly stressed; a lying suspect may crack and reveal a new fact, an honest one pushes back.

5. **Accuse** — at any time, name the killer and attach evidence. This ends the game (win or lose). One accusation only.

2.2 The Clock

The night is the difficulty system: **30 actions** total ("the sun comes up"). Questions and presses both consume actions, so the player must choose between breadth (questioning everyone) and depth (pressuring one liar). The remaining actions are always visible as a burning cigarette that shortens as dawn nears.

2.3 What the Player Sees

- A chat window per suspect (dialogue, tone, body-language "tells" rendered as italic stage directions, e.g., *she glances at the door*).
- The **notebook panel**: bulleted facts with source attribution ("Marco: claims he was at the bar until 1:30"), contradictions highlighted in red.
- An **evidence tray** of 6 physical items found at the scene (established at case generation), drag-and-droppable into a Press or an Accusation.
- The **stress meter** on each portrait (calm → rattled → cracking).

2.4 Why It's a Game and Not a Chatbot

The player cannot brute-force the answer by chatting forever: actions are scarce, suspects only crack under a *correct* press (wrong evidence wastes an action and hardens them), and the win requires evidence, not a hunch. The fun is deduction under time pressure — the AI improvises the *texture*, but the underlying case logic is fixed and fair.

---

3. AI Architecture (what each agent does, through its effect on gameplay)

Four named agent roles. One runs before the game; three run during play.

3.1 Case Architect Agent (pre-game, runs once per new case)

Development role in one sentence: Generates the complete hidden case file — killer, timeline, alibis, six evidence items, and three planted contradictions — as a single validated JSON document.

Gameplay effect: Replayability. Every new game is a genuinely different whodunit with a solvable logic structure. The case file is the ground truth that all other agents obey; the player never sees it directly.

Output format: `case_file.json` — `{killer_id, timeline[], suspects[5]{id, persona, true_whereabouts, public_alibi, secret, knows[]}, evidence[6]{id, description,`

`implicates}, contradictions[3]{stmt_a, stmt_b, reveals}}`.

3.2 Suspect Agent (runtime — 5 instances, one per character)

Development role in one sentence: Answers the player's questions in character, using only that character's knowledge slice from the case file, lying only where its `secret` requires it.

Gameplay effect: This *is* the game — the conversations the player interrogates. Each instance receives only its own knowledge (not the killer's identity, unless it is the killer), so an agent cannot leak what its character could not know.

Output format: JSON per turn — `{dialogue (max 60 words), tone (one of: calm, defensive, evasive, hostile, cracking), stress_delta (-1..+2), tell (one short stage direction or null)}`.

3.3 Consistency Warden Agent (runtime — after every suspect reply)

Development role in one sentence: Checks each suspect reply against the case file and that suspect's previous statements, and rewrites the reply if it contradicts ground truth *unintentionally*.

Gameplay effect: Fairness. Deliberate lies (from `secret`) pass through; hallucinated facts that would make the mystery unsolvable get corrected before the player ever sees them. The player experiences suspects who lie for

reasons, not from model drift.

Output format: `{verdict: "pass" | "violation", corrected_dialogue | null, violated_fact_id | null}`.

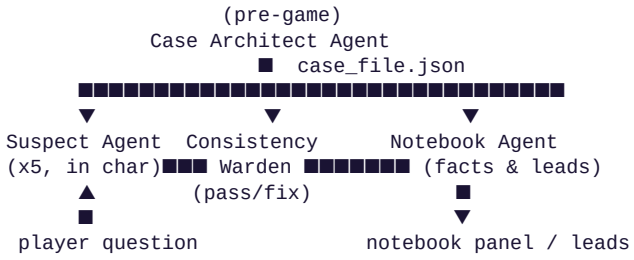
3.4 Notebook Agent (runtime — after every player-visible reply)

Development role in one sentence: Extracts new factual claims from the conversation, appends them to the notebook, and detects when two recorded claims contradict each other.

Gameplay effect: The deduction interface. When it detects a clash between two testimonies it highlights both lines in red and mints a **Lead** the player can use in a Press. It never invents facts — it only indexes what suspects actually said.

Output format: `{new_facts[]{suspect_id, claim, turn_ref}, contradiction | null {fact_a, fact_b, lead_text}}`.

3.5 Agent Interaction Diagram



4. Technical Strategy

4.1 Stack and Scope

- **Backend:** Python 3.12, FastAPI, Anthropic SDK. Game state (case file, conversation logs, notebook, action count) held server-side in a single session object; no database needed for the prototype.
- **Frontend:** One static HTML/JS page — chat window, portraits, notebook panel, evidence tray. No engine, no build step.
- **Art:** 5 static AI-generated portraits + 1 club background, generated once during development (not at runtime).
- **Shipped scope:** 1 hand-tuned seed case + the Case Architect generator, 5 suspects, 6 evidence items, 30-action night. Nothing else.

4.2 Model Assignment and Token Budget

Agent	Model	Calls / session	Tokens in / call	Tokens out / call	Session total
Case Architect	Claude Sonnet	1	2,500	4,000	6,500
Suspect	Claude Haiku	~30	1,800 (persona + window)	150	58,500
Consistency Warden	Claude Haiku	~30	1,500 (reply + relevant facts)	100	48,000
Notebook	Claude Haiku	~30	900	120	30,600
Total (typical 30-action night)					≈ **144,000 tokens**

Estimated cost per full playthrough at current API pricing: ≈ **$0.25–0.35** (Haiku for all runtime calls; Sonnet only for one-time case generation). A **hard cap of 250,000 tokens per session** aborts to a graceful "the trail went cold" ending if ever reached (roughly double the typical night, so honest players never see it).

4.3 API Constraints and How the Design Absorbs Them

- **Latency:** Every player-visible reply requires Suspect + Warden + Notebook calls. Warden and Notebook run concurrently after the Suspect call; target ≤ 3 s perceived latency using Haiku, with a typing indicator ("she takes a drag before answering") covering the gap.
- **Hidden-information leakage:** Mitigated structurally, not by prompting alone — each Suspect Agent's context contains only its own knowledge slice, so the model *cannot* reveal the killer by accident.
- **Context growth:** Conversation windows are trimmed to the last 12 exchanges per suspect; older statements live in the notebook (the ground truth for contradiction checks), keeping per-call input flat over a session.

4.4 Named Constraint (Feasibility)

Constraint: per-session API cost and latency ceiling. As a solo developer with a personal API key, I cannot afford Sonnet-class models on every conversational turn, and three sequential model calls per reply would feel sluggish. This is why runtime agents run on Haiku, the Warden/Notebook calls are parallelized, output formats are capped (60-word replies), and the 30-action clock exists — the core difficulty mechanic doubles as the token budget enforcer. If Haiku's in-character quality proves insufficient during Week 1 testing, the fallback is pre-generating richer persona sheets with Sonnet at case-creation time while keeping runtime on Haiku.

4.5 Development Plan (remaining weeks)

1. **Week 1:** Case file schema + Case Architect + 1 seed case; CLI-only interrogation of one suspect.
2. **Week 2:** All 5 suspects + Consistency Warden + action clock; win/loss.
3. **Week 3:** Notebook Agent, leads, press mechanic; browser UI.
4. **Week 4:** Balance pass (agent review crew from Class 03), portraits, polish, submission build.